

ADC-Aware Post-Training Adaptation for Analog LLM Inference under Fixed Output Quantization

Anonymous submission

Abstract

Analog in-memory and optical matrix–vector accelerators promise substantial energy savings for large language model (LLM) inference, but they introduce an output-side error absent from purely digital deployment. After each analog accumulation, a finite-resolution analog-to-digital converter (ADC) maps the result onto a hardware-fixed lattice, adding a coarse quantization step on top of the usual weight and activation quantization. Crucially, unlike the weight and activation step sizes that calibration is free to tune, this ADC step is a hardware constant fixed by tile width, ADC precision, and range scaling. Standard post-training quantization (PTQ) can thus reshape the values fed into the ADC, but it cannot make the ADC lattice itself any finer. Existing analog-aware methods address this with quantization-aware training or full finetuning, which requires end-to-end gradient training of the entire model with the hardware in the loop and must be repeated for each model and hardware configuration—prohibitively expensive at LLM scale. We instead ask how far a pretrained decoder-only LLM can be adapted to this ADC-constrained path by post-training methods alone.

We adapt and evaluate Llama-3.2-1B under this ADC-constrained path on WikiText-2 and C4 perplexity and standard downstream benchmarks. A naive PTQ model is unusable—W4A4 WikiText-2 perplexity collapses to 2354.86—so making it work requires two ingredients. ADC-aware calibration built on FlatQuant (propagated hidden states, a mixed clean/ADC objective, and bounded diagonal scaling under a staged schedule) first brings perplexity down to 26.66. A rank-4 post-ADC LoRA correction, trained with the analog backbone frozen, then reduces WikiText-2/C4 ADC perplexity to 14.03/23.20 using only 0.15% extra parameters; the same recipe carries over to a hardware-realistic unsigned shift-subtract readout.

To our knowledge, this is the first fully post-training pipeline that turns an otherwise-collapsed ADC-constrained LLM into a working model, showing that fixed low-resolution ADCs can be handled by lightweight adaptation on the digital side of the analog–digital boundary rather than by full retraining.

Introduction

Large language models are dominated by dense linear projections. During autoregressive decoding, each generated token requires repeated reads of large weight matrices, so inference is often limited by memory traffic and data movement rather than raw arithmetic throughput. In-memory

computing (IMC) and optical accelerators address this bottleneck by executing matrix–vector multiplication (MVM) directly in the physical substrate that stores the weights. The analog array performs the accumulation, but the next transformer operation still expects a digital tensor.

That readout stage is the central focus of this paper. In a standard digital low-bit model, the dominant approximation error comes from quantizing weights and activations. In an analog deployment path, there is an additional output-side quantizer: after each analog accumulation, a finite-resolution ADC maps the result onto a discrete lattice before the next layer can use it. Formally, if x_q and W_q denote the activation and weight integer codes, the integer accumulation $y_{\text{int}} = W_q x_q$ is mapped as

$$y_{\text{adc}} = \Delta \cdot \text{clip}\left(\left\lfloor \frac{y_{\text{int}}}{\Delta} \right\rfloor, n_a, p_a\right), \quad (1)$$

where (n_a, p_a) are the output-code bounds and the step size is

$$\Delta = \frac{2 M q_x q_w}{2^{b_a} k}, \quad (2)$$

with tile width M , ADC bit-width b_a , range-scaling factor k , and input/weight code ranges q_x, q_w . Crucially, Δ is not a learnable software quantity: every term in Eq. (2) is fixed by hardware. Therefore, even if weights and activations are quantized well, the accumulated output can still be rounded into a coarse lattice, clipped, or driven into a dead zone (Figure 1).

This failure mode is especially severe for LLMs. Transformer hidden states contain rare but very large activation outliers. Under max-based per-token scaling, a single outlier channel can determine the activation scale and compress typical channels into a narrow band of integer codes. Once those codes are accumulated tile by tile and passed through a coarse ADC lattice, output bins are poorly used and small accumulations collapse to zero. Standard LLM PTQ methods reduce input-side quantization error, but they are not designed to optimize the post-accumulation ADC output.

Recent analog-aware methods such as RAOQ and Analog Foundation Models confirm that analog deployment cannot be solved by digital quantization alone, but they operate in a more training-intensive regime that we discuss in the next section. Our question is narrower: starting from an already trained decoder-only LLM, and assuming full analog-aware

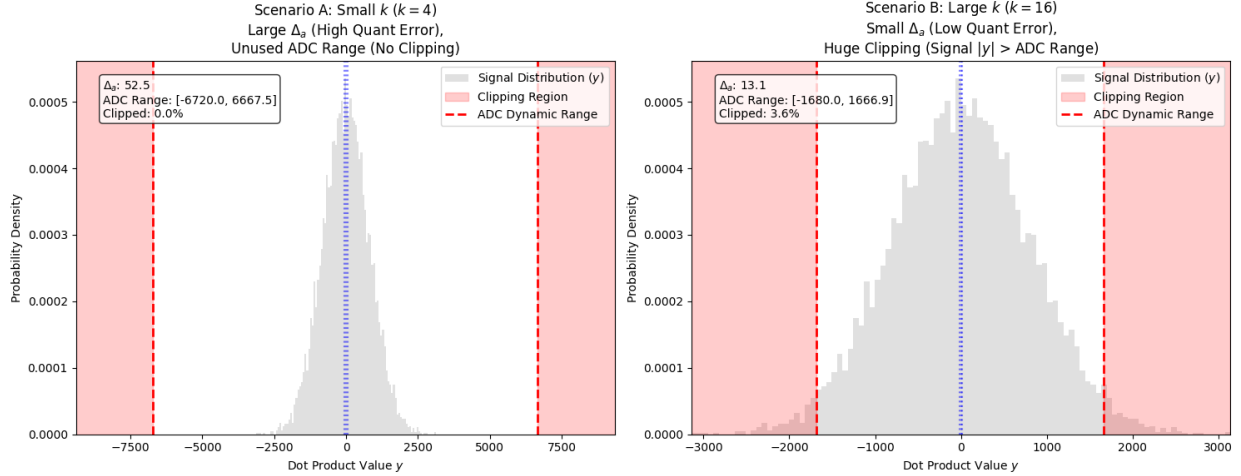


Figure 1: The ADC range–resolution tradeoff set by the fixed scaling factor k (and hence Δ). Small k (left) gives a large step Δ : the accumulation distribution fits inside the ADC range with no clipping, but the lattice is coarse. Large k (right) shrinks Δ for finer resolution, yet the tails of the distribution now exceed the ADC range and are clipped. Because k is fixed in hardware, calibration alone cannot escape this tradeoff.

finetuning is undesirable or unavailable, how far can post-training adaptation go?

Taken together, these observations define the problem we tackle. The ADC is a fixed, output-side quantizer that PTQ cannot sharpen; its damage accumulates block by block; and the transforms normally used to fight outliers have limited reach. We address all three with a fully post-training recipe that combines ADC-aware calibration *before* the converter with a lightweight digital correction *after* it. This separation between pre-ADC calibration and post-ADC correction is the key conceptual distinction of our method.

Our contributions follow directly from these problems. **First**, we formulate Llama-style inference under a tiled ADC-constrained MVM model and isolate ADC-induced error from ordinary quantization error using bypass evaluation and per-layer diagnostics. **Second**, we develop an ADC-aware PTQ pipeline—propagated calibration, an α -mixed objective, and bounded diagonal scaling under a staged schedule—backed by a lemma showing that orthogonal Kronecker transforms have an intrinsic ceiling on outlier suppression, which motivates the diagonal scaling. **Third**, we introduce a post-ADC rank-4 residual LoRA correction that, with the analog path frozen, reduces WikiText-2/C4 ADC perplexity to 14.03/23.20 and surpasses our INT8 PTQ baseline. **Finally**, we show that the same recipe transfers to a hardware-realistic unsigned shift-subtract readout, where per-layer ADC range search and post-ADC correction are complementary.

Background and Related Work

ADC-Constrained Analog Inference

Analog in-memory and optical accelerators reduce data movement by executing MVM directly in the substrate that stores the weights (Xiao et al. 2021; Spall, Guo, and Lvovsky 2022; Le Gallo et al. 2023). Weights are mapped to

physical states, the input is applied as an analog excitation, and array physics performs the accumulation. Because the result is analog while the next transformer operation expects a digital tensor, each MVM must be followed by an ADC.

This output quantizer differs fundamentally from standard digital quantization. First, it is applied *after* accumulation, when information from many channels has already been mixed. Second, its step size is fixed by hardware parameters, so a software method can improve scale selection but cannot arbitrarily reduce the lattice spacing. As a result, analog output quantization is a distinct source of error rather than a minor implementation detail.

Recent analog-aware methods make this point explicit. RAOQ addresses output quantization through a dedicated training stage after QAT (Tang et al. 2024), while Analog Foundation Models adapts pretrained LLMs to noisy low-precision analog hardware using a broader hardware-aware training pipeline (Buechel et al. 2025). We build on the same diagnosis—that ADC/output quantization is a first-class bottleneck—but pursue a lighter-weight regime based on post-training calibration and a small digital correction module.

Low-Bit LLM Quantization

Quantization maps real-valued tensors to low-bit integer codes to reduce memory, bandwidth, and arithmetic cost (Gholami et al. 2021; Jacob et al. 2018). In symmetric uniform quantization, values are scaled, rounded, clipped, and dequantized back to floating point. PTQ calibrates these operations on a small dataset; QAT places quantization inside the optimization loop and typically relies on the straight-through estimator for non-differentiable rounding (Esser et al. 2019).

LLMs are particularly difficult to quantize because their hidden states are anisotropic and outlier-heavy.

SmoothQuant migrates dynamic range from activations into weights through an equivalent diagonal transform (Xiao et al. 2022). Outlier Suppression+ uses channel-wise shifting and scaling (Wei et al. 2023). AWQ protects salient weights selected from activation statistics (Lin et al. 2023), OmniQuant combines learnable clipping with equivalent transformations for very low-bit PTQ (Shao et al. 2023), and GPTQ frames weight quantization as a second-order reconstruction problem (Frantar et al. 2022). Rotation-based methods such as QuaRot and SpinQuant suppress residual-stream outliers with fixed or learned rotations (Ashkboos et al. 2024; Liu et al. 2024b), while FlatQuant learns invertible per-projection Kronecker transforms and folds them into the weights (Liu et al. 2024a). CBQ and quantization-error-propagation PTQ move beyond local block reconstruction by modeling how quantization error accumulates across depth (Ding et al. 2023; Anonymous 2025). These methods improve digital quantization, but they do not directly optimize the separate ADC output lattice.

Lightweight Post-Quantization Adaptation

When pure PTQ saturates, a natural next step is to freeze the quantized backbone and attach a small trainable correction. Prior work on low-bit transformers has shown that aggressive quantization often benefits from finetuning or QAT-style recovery (Liu et al. 2023; Chen et al. 2024). Low-rank methods such as LoRA and related quantization-error reconstruction schemes capture much of the residual error with only a small number of trainable parameters (Hu et al. 2022; Zhang et al. 2024).

For ADC-constrained inference, this family of methods is particularly attractive. A frozen quantized analog path can preserve the hardware mapping, while a lightweight digital residual branch learns the remaining distortion. What is non-standard in our setting is the placement of the correction: not before quantization, but *after* the ADC, on the digital side of the analog–digital boundary. In short, prior work reduces input-side error effectively but leaves the output-side ADC lattice largely unaddressed, and existing analog-aware remedies rely on hardware-aware training; we instead ask how much of that residual loss post-training adaptation alone can remove.

Problem Statement

We consider a pretrained decoder-only transformer whose linear projections are executed on a tiled ADC-constrained MVM accelerator. For each projection $y = xW^\top$, activations are quantized per token and weights per output channel. The integer accumulation is computed tile-by-tile, quantized by the fixed ADC of Eq. (1), and dequantized by the activation and weight scales (Figure 2). The effective layer map is

$$\hat{f}_\ell^{\text{ADC}}(x) = D_{\text{out}} Q_{\text{ADC}}(Q_w(W_\ell) Q_x(x)). \quad (3)$$

Here Q_x and Q_w are the input-side quantizers, Q_{ADC} is the fixed output lattice of Eq. (1), and D_{out} absorbs the activation and weight scales. Our goal is to preserve model quality under this constraint without retraining the full model. This

raises a natural progression of questions: how far transform-based PTQ can mitigate the ADC, whether reconstruction-only calibration is sufficient or an ADC-aware objective is required, and when adaptation beyond PTQ becomes necessary—which the rest of the paper answers in turn. The study uses a software ADC simulator that models finite-resolution output quantization, tiling, and integer arithmetic while abstracting device-level non-idealities.

ADC-Aware Post-Training Adaptation

We build the method in four PTQ steps—a FlatQuant baseline, ADC-propagated calibration with a mixed objective, bounded diagonal scaling, and a staged schedule—and then add a post-ADC residual branch for the error PTQ provably cannot reach.

Baseline: FlatQuant for Llama

FlatQuant exploits a simple linear invariance of the MVM. For any invertible T ,

$$y = xW^\top = (xT^{-1})(TW^\top), \quad (4)$$

so choosing T to flatten the distribution of xT^{-1} improves both integer quantization and ADC bin coverage while preserving the map. A full $d \times d$ transform is prohibitive, so FlatQuant parameterizes $T = T_1 \otimes T_2$, reducing the number of trainable parameters from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$ (Liu et al. 2024a).

Calibration is teacher–student. A frozen FP teacher produces y^* , and the quantized student minimizes $\|\hat{y} - y^*\|_2^2$ over the Kronecker factors; after calibration, T is folded into the weights at no inference cost. We wrap all seven projections of each decoder block (q, k, v, o in grouped-query attention and `gate`, `up`, `down` in the SwiGLU FFN). Standard block calibration uses clean teacher activations,

$$\mathcal{L}_{\text{FP}} = \left\| \hat{B}(x^*) - B_{\text{FP}}(x^*) \right\|_2^2, \quad (5)$$

which is well matched to digital PTQ but mismatched to analog deployment, where each block receives activations already corrupted by prior ADC readouts.

Cross-Block Propagation and α -Mixing

To match deployment, blocks are calibrated sequentially from layer 0 to $L - 1$. When calibrating block ℓ , its input is the ADC-quantized output of the already calibrated block $\ell - 1$. Thus the block sees the same accumulated distortion during calibration that it will see at inference:

$$\mathcal{L}_{\text{ADC}} = \left\| \hat{B}(\hat{x}) - B_{\text{FP}}(x^*) \right\|_2^2, \quad (6)$$

where $\hat{x}$ is the ADC-propagated hidden state. Full propagation is noisy in W4A4 because gradients must pass through several floor operators via STE. We therefore interpolate between the clean and propagated objectives,

$$\mathcal{L}(\alpha) = (1 - \alpha) \mathcal{L}_{\text{FP}} + \alpha \mathcal{L}_{\text{ADC}}. \quad (7)$$

The clean term stabilizes the transform, while the propagated term teaches robustness to ADC distortion. A sweep

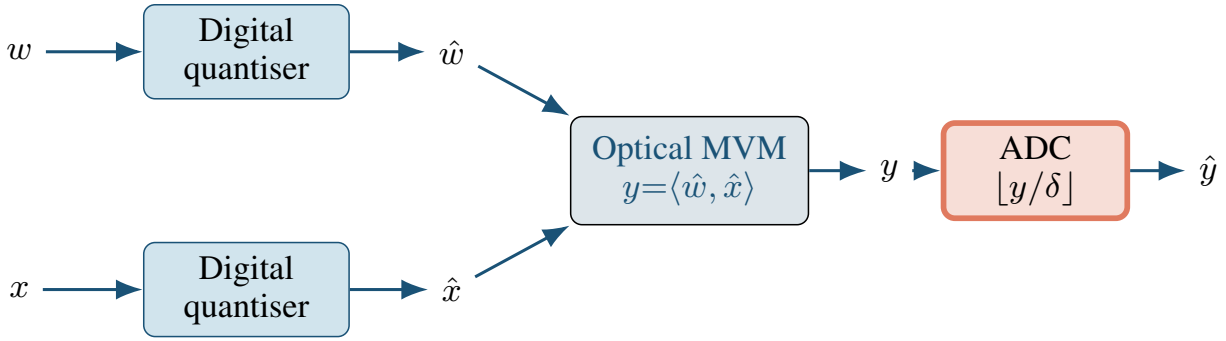


Figure 2: Math model of an optical linear layer. Weights w and activations x are digitally quantized to $\hat{w}, \hat{x}$, multiplied in the analog (optical) matrix–vector unit, and the accumulation y is mapped back to the digital domain by a fixed-resolution ADC, $\hat{y} = \Delta \lfloor y/\Delta \rfloor$. The output-side ADC step is set by hardware and is not a free calibration variable.

in the appendix shows that $\alpha = 0.5$ is the best W4A4 point: below it the ADC regime is under-weighted, above it floor-induced gradient noise degrades transform quality. At INT8, propagation alone drops ADC perplexity from 28.86 to 14.55 with no extra parameters.

Trainable Per-Channel Diagonals

Propagation and α -mixing fix *what* the calibration optimizes, but the transform itself still has limited capacity to tame outliers: rotations redistribute energy across channels without rescaling any of them. We first make this precise.

Lemma 1 (Rotation-only outlier suppression bound). *Let $P = L \otimes R$ with $L^\top L = I$, $R^\top R = I$ be an orthogonal Kronecker transform on $\mathbb{R}^d$, and define the effective participation ratio $m_{\text{eff}}(x) = \|x\|_2^2 / \|x\|_\infty^2 \in [1, d]$. Then for every such P ,*

$$\|xP\|_\infty \geq \frac{\|x\|_2}{\sqrt{d}}, \quad \frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \leq \sqrt{\frac{d}{m_{\text{eff}}(x)}}. \quad (8)$$

The proof (appendix) follows from orthogonal invariance of $\|\cdot\|_2$ and the ℓ^∞ – ℓ^2 inequality. With $d = 2048$ and 8 dominant channels ($m_{\text{eff}} \approx 8$), the peak can be suppressed by at most a factor 16. If more suppression is required, rotation alone is provably insufficient. Moreover, the Kronecker family is a strict low-dimensional subset of all orthogonal transforms (2512 vs. $\approx 2.1 \times 10^6$ free parameters for $d = 2048$), so the true reachable suppression can be even smaller.

We therefore absorb a trainable diagonal $D = \text{diag}(d_1, \dots, d_c)$, $d_i \in [10^{-4}, 10]$, into the transform, preserving the map by equal-and-opposite rescaling of the weights,

$$y = (x D^{-1} T^{-1})(T D W^\top). \quad (9)$$

Because D is not orthogonal, it breaks the ℓ^2 -energy invariance of Lemma 1. Setting $d_j < 1$ reduces the contribution of an outlier channel *before* the per-token scale is computed, achieving suppression no rotation can match. The bounds prevent degenerate scaling, and D folds into the weights at deployment. We place separate diagonals on the attention path (around $q/k/v$ and o) and the MLP path (around

gate/up and down). Ablations show that they play different roles: MLP diagonals drive no-ADC transform quality, while attention diagonals close the ADC gap.

Staged Calibration

With the objective and the transform family in place, the last question is how to optimize them together. Training all Kronecker factors and both diagonals jointly with $\alpha = 0.5$ is unstable: attention gradients dominate before MLP transforms converge. We therefore split calibration into two stages. *Stage A* trains the MLP Kronecker factors and MLP diagonals with clean reconstruction ($\alpha = 0$), giving a stable warm start. *Stage B* freezes the MLP diagonals, unfreezes the attention Kronecker factors and attention diagonals, and fine-tunes with $\mathcal{L}(0.5)$ at a lower learning rate. This schedule avoids gradient interference and gives the best pure-PTQ checkpoint (bypass 18.50, ADC 27.60).

Algorithm 1 makes the procedure precise over the projection set $\mathcal{P} = \{q, k, v, o\} \cup \{\text{gate}, \text{up}, \text{down}\}$, with $\text{attn} = \{q, k, v, o\}$ and $\text{MLP} = \{\text{gate}, \text{up}, \text{down}\}$. Gradients through the quantizers Q_x, Q_w, Q_{ADC} use the straight-through estimator (STE).

Post-ADC Residual LoRA

Even the best staged checkpoint still leaves an ADC gap, and this ceiling is fundamental: pure PTQ can only reshape the signal *before* the ADC. Once the ADC maps a continuous accumulation to a discrete code, all sub-step information inside that bin is gone from the frozen analog path. This is why pure PTQ plateaus around 27.5–28.5 W4A4 ADC perplexity across calibration strategies. We add a small digital residual branch *after* the ADC (Figure 3). For a projection with frozen ADC output, the adapted output is

$$y = \text{ADC}_{\text{frozen}}(\hat{x}) + \frac{s}{r} B(A x_{\text{fp}}), \quad (10)$$

with $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $\text{rank } r = 4$, and LoRA scale $s = 8$, all computed in float32 (we reserve α for the calibration mixing coefficient). The entire base—weights, Kronecker factors, diagonals, and ADC steps—stays frozen. Rank-4 adapters on all seven projections of 16 blocks add $\sim 1.8\text{M}$ parameters (0.15% of the model). The adapters are

Algorithm 1: ADC-Aware Staged PTQ Calibration

Input: FP model F with teacher blocks $\{B_\ell^{\text{FP}}\}_{\ell=1}^L$; calibration set C ; hardware $H = (M, b_a, k, q_x, q_w)$ fixing the ADC step Δ ; base lr η ; epochs E_A, E_B ; box $[\underline{d}, \bar{d}] = [10^{-4}, 10]$

Output: Quantized ADC-aware model $\hat{F}_{\text{ADC}}$

```

1: Def.  $\tilde{T}_p := T_p D_p$ ; per projection  $p$ ,
    $\hat{f}_p(x) = D_{\text{out}} Q_{\text{ADC}}(Q_w(\tilde{T}_p W_p^\top) Q_x(x \tilde{T}_p^{-1}))$ ;
    $\hat{B}_\ell(\cdot; \theta)$  composes  $\{\hat{f}_p\}$  over  $p \in \mathcal{P}$ 
2: for block  $\ell = 1$  to  $L$  do
3:    $x^* \leftarrow$  clean input;  $y^* \leftarrow B_\ell^{\text{FP}}(x^*)$  (teacher)
4:    $\hat{x} \leftarrow$  ADC-propagated output of calibrated block  $\ell-1$ 
5:   init  $T_p = T_{p,1} \otimes T_{p,2}$ ,  $D_p = I$ ,  $\forall p \in \mathcal{P}$ 
6:   Stage A ( $\alpha=0$ ):  $\theta_A = \{T_p\}_{p \in \text{MLP}} \cup \{D^{(m)}\}$ 
7:   for  $e = 1$  to  $E_A$ ; minibatch  $\subset C$  do
8:      $\mathcal{L}_{\text{FP}} = \|\hat{B}_\ell(x^*; \theta) - y^*\|_2^2$  (Eq. 5)
9:      $\theta_A \leftarrow \text{AdamW}(\theta_A, \nabla_{\theta_A} \mathcal{L}_{\text{FP}}; \eta)$  (STE)
10:     $D^{(m)} \leftarrow \text{clip}(D^{(m)}, \underline{d}, \bar{d})$ 
11:   end for
12:   freeze  $D^{(m)}$ ; lr  $\leftarrow 0.1 \eta$ 
13:   Stage B ( $\alpha=0.5$ ):  $\theta_B = \{T_p\}_{p \in \text{attn}} \cup \{D^{(a)}\}$ 
14:   for  $e = 1$  to  $E_B$ ; minibatch  $\subset C$  do
15:      $\mathcal{L}_{\text{ADC}} = \|\hat{B}_\ell(\hat{x}; \theta) - y^*\|_2^2$  (Eq. 6)
16:      $\mathcal{L}(0.5) = \frac{1}{2} \mathcal{L}_{\text{FP}} + \frac{1}{2} \mathcal{L}_{\text{ADC}}$  (Eq. 7)
17:      $\theta_B \leftarrow \text{AdamW}(\theta_B, \nabla_{\theta_B} \mathcal{L}(0.5); 0.1 \eta)$ 
18:      $D^{(a)} \leftarrow \text{clip}(D^{(a)}, \underline{d}, \bar{d})$ 
19:   end for
20:   fold  $W_p^\top \leftarrow \tilde{T}_p W_p^\top$ , fuse  $\tilde{T}_p^{-1}$  into input side; store
      $Q_w(\cdot)$ , fixed  $\Delta$  (Eq. 9)
21: end for
22: return  $\hat{F}_{\text{ADC}}$ 

```

trained with next-token cross-entropy plus teacher distillation,

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p}_T \| p_{\text{FP}, T}), \quad (11)$$

with $\lambda = 0.5$ and temperature $T = 2$. Post-ADC placement is essential. A pre-ADC adapter is itself quantized by the fixed lattice, cannot express sub- Δ corrections, and diverges (ADC perplexity ≈ 105.6). Post-ADC, the correction is added in the digital domain at full precision with clean gradients, dropping W4A4 ADC perplexity from 27.60 to 14.03.

Experiments

Setup

Experiments use Llama-3.2-1B in bfloat16 (Dubey et al. 2024): 16 decoder layers, hidden dimension 2048, grouped-query attention, and a SwiGLU FFN with intermediate dimension 8192. The model is small enough to sweep dozens of configurations overnight on a single GPU while remaining representative of the Llama family.

Calibration and LoRA use WikiText-2 training text (Merity et al. 2017). We evaluate WikiText-2 and C4 perplexity (Raffel et al. 2020) with context length 2048 and stride 1024,

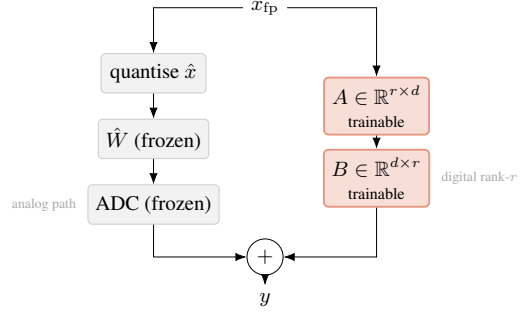


Figure 3: Post-ADC residual LoRA. The quantized analog path (weights, transforms, diagonals, and ADC) stays frozen; a trainable rank- r branch $B(A x_{\text{fp}})$ is added *after* the ADC in the digital domain, correcting the residual error the fixed lattice cannot express.

and downstream accuracy via the lm-evaluation-harness (Gao et al. 2024). The main hardware setting uses tile size $M = 256$, ADC bitwidth $b_a = 8$, scaling factor $k = 16$, and W4A4 or W8A8 precision. This gives a signed step $\Delta \approx 2016$ for W8A8 and $\Delta \approx 6.12$ for W4A4.

We report two perplexities. *Bypass* disables the ADC and isolates tiling plus integer quantization. *ADC* enables the full pipeline and is the primary metric; the gap between the two measures ADC-induced degradation. Internal diagnostics (dead-rate, clip-rate, bin usage, reconstruction error) are defined in the appendix. FlatQuant transforms train for 30 epochs at 5×10^{-3} (Stage B: 10 epochs at 5×10^{-4}); post-ADC LoRA trains for 5 epochs at 10^{-4} .

Main Results

Table 1 reports the main results. Digital INT8 is nearly loss-less, but enabling the ADC sharply degrades quality. The effect is stronger at INT4: the digital baseline is still usable, while the ADC-enabled model remains far worse even after ADC-aware PTQ. Without ADC-aware calibration, the naive W4A4 ADC baseline collapses to 2354.86 WikiText-2 perplexity; ADC-aware PTQ reduces this to 26.66 (a factor of ≈ 88), but still remains far from the bfloat16 baseline of 8.68.

Post-ADC LoRA closes much of this remaining gap. It reduces WikiText-2/C4 ADC perplexity from 26.66/45.62 to 14.03/23.20, closing roughly 70% of the remaining WikiText-2 gap and 69% of the C4 gap, and surpassing INT8 PTQ (16.85). Downstream accuracy improves from 41.99 to 47.07. The largest gains appear on commonsense and science tasks (HellaSwag, ARC-Easy, PIQA, OpenBookQA), while MMLU and WinoGrande improve less. This suggests that next-token CE+KL transfers well to perplexity, but only partially to all multiple-choice tasks. Per-task numbers are in the appendix.

Ablation Summary

We summarize the key findings here; full tables are in Appendices C–D. At INT8, propagation is the dominant lever (ADC 28.86 $\rightarrow$ 14.55), while a bin-center auxiliary loss

Method	WikiText-2 PPL	C4 PPL	Avg. downstream acc.
bfloat16	8.68	13.13	54.25
INT8 PTQ, ADC disabled	8.69	13.15	55.95
INT8 + ADC PTQ	16.85	29.56	44.97
INT4 PTQ, ADC disabled	14.13	22.83	48.64
INT4 + ADC PTQ	26.66	45.62	41.99
INT4 + ADC + post-ADC LoRA	14.03	23.20	47.07

Table 1: Main Llama-3.2-1B results. Lower perplexity is better; higher downstream accuracy is better. The post-ADC LoRA row keeps the quantized ADC backbone frozen and trains only rank-4 residual adapters.

adds only 0.14. At INT4, full propagation is too noisy and $\alpha = 0.5$ is best. Sample count is also important: the naive $\times 153$ bypass-ADC gap at 128 samples closes to $\times 1.2$ at 512 samples with propagation.

Diagonal placement matters. MLP-only scaling gives strong bypass quality but a wide ADC gap; attention-only scaling helps the ADC gap but hurts bypass; staged MLP-then-attention training is best overall (bypass 18.50, ADC 27.60). Within the MLP, the down diagonal drives nearly all of the improvement. Stochastic propagation (Bernoulli, Beta) does not beat deterministic $\alpha = 0.5$.

For post-ADC LoRA, rank saturates at $r = 4$. CE+KL beats CE-only, especially when covering all seven projections (by 2.65 perplexity) and for C4 transfer (C4 ADC 46.40 $\rightarrow$ 23.30 despite WikiText-2-only training). Covering all 16 layers beats last-8 (15.63 vs. 20.08), confirming that ADC error accumulates with depth.

Unsigned Shift-Subtract Readout

The signed readout is a useful reference, but many physical crossbars accept only non-negative currents. We therefore evaluate an unsigned shift-subtract readout. Signed INT4 codes are shifted to $[0, 15]$, a single non-negative MVM is performed, and deterministic zero-point terms are subtracted digitally after ADC readout (exact recovery; derivation in the appendix). The unsigned step at global $k = 16$ is

$$\Delta_u = \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{2^{b_a k}} \approx 14.06, \quad (12)$$

more than twice the signed W4A4 step. Because this coarser step imposes a heavier resolution penalty, we add a per-layer search over $k \in \{4, 8, 16, 32, 64\}$. For each projection, the search selects the smallest k that avoids ADC saturation on the calibration set, and it runs in place on the frozen PTQ checkpoint.

Table 2 shows that the global unsigned baseline (39.00) is much worse than the signed reference. Per-layer k search alone reduces it to 21.64 without retraining; post-ADC LoRA reaches 17.29; and the combination reaches 14.56. Thus range adaptation before deployment and residual correction after the ADC are complementary, and both transfer to the more realistic unsigned readout.

Discussion

Three conclusions follow. First, ADC error is not the same as digital quantization error. A no-ADC evaluation can make a

Unsigned configuration	Wiki PPL	C4 PPL
bfloat16	8.68	13.13
INT4 FlatQuant, ADC off	12.56	20.13
Unsigned PTQ, global $k = 16$	39.00	67.78
+ per-layer k	21.64	33.79
+ post-ADC LoRA	17.29	29.26
+ per-layer k + LoRA	14.56	23.78

Table 2: Unsigned shift-subtract results. Per-layer k search and post-ADC LoRA are complementary.

low-bit LLM appear healthy while hiding a major hardware-induced degradation, so the ADC must be evaluated inside the inference loop.

Second, ADC-aware calibration is mainly a distribution-matching problem. Clean teacher activations do not match what deeper blocks see at analog deployment time. Propagating ADC-quantized states exposes blocks to their real inference distribution, but the objective must be mixed with a clean term to avoid optimizing only noisy floor-induced gradients.

Third, pure PTQ has a natural ceiling because it acts before the lossy ADC. Transforms, rotations, and diagonals can place the pre-ADC signal in a better range, but they cannot recover sub-step information once the output code is selected. A post-ADC residual branch works precisely because it operates on the digital side of the boundary, making it qualitatively different from another pre-ADC transform. The results also suggest a hardware-software co-design direction: tile size, ADC bitwidth, scaling factor, signedness, and calibration objective jointly determine accuracy, and modest configurability such as per-layer k is valuable when exposed to the calibration pipeline.

Limitations

This study uses a software ADC simulator rather than measurements on fabricated hardware. Real devices may add non-uniform bins, offset, drift, thermal noise, IR drop, and latency constraints; the simulator isolates finite-resolution output quantization but does not replace hardware validation. The experiments focus on Llama-3.2-1B, and larger models may show different outlier statistics or require different calibration budgets. The adaptation data is WikiText-2, with C4 as transfer evidence, so broader domains should be

evaluated. Finally, the post-ADC LoRA branch has nonzero digital overhead and requires access to the adapter input; a hardware implementation must decide whether it runs on a digital co-processor, is restricted to selected projections, or is fused into a larger post-processing path.

Conclusion

Finite-resolution ADCs introduce an output-side quantization bottleneck that standard digital PTQ does not solve. For Llama-3.2-1B under W4A4 analog inference, naive ADC-enabled FlatQuant collapses, while ADC-aware PTQ with propagated calibration, mixed objectives, bounded diagonals, and staged training restores usable quality. The remaining gap is not removable by pre-ADC calibration alone: a rank-4 digital correction placed after the ADC recovers most of it while keeping the quantized backbone frozen, and the same recipe transfers to a hardware-realistic unsigned readout. The broader message is that the analog-digital boundary is an algorithmic object — accurate analog LLM inference requires treating the ADC as part of the quantization pipeline, not as a passive hardware detail.

A. Detailed Configurations and Metrics

Hardware constants. Table 3 lists the fixed ADC constants used throughout the signed experiments. The step size is fully determined by $\Delta = 2Mq_xq_w/(2^{b_a}k)$, giving $\Delta \approx 2016$ for INT8 and $\Delta \approx 6.12$ for INT4.

Parameter	Symbol	Value
Tile size (crossbar columns)	M	256
ADC bit-width	b_a	8
Parallel ADC channels	k	16
INT8 weight/activation range	q_w, q_x	127
INT4 weight/activation range	q_w, q_x	7
ADC step (INT8)	Δ	≈ 2016
ADC step (INT4)	Δ	≈ 6.12

Table 3: Fixed ADC hardware parameters for the signed bipolar readout used in the main experiments.

Metrics. Beyond bypass and ADC perplexity, we track four internal diagnostics on the calibration set. **dead_rate**: fraction of output channels with $|y_{\text{int}}| < \Delta$, i.e. the ADC output is identically zero (lower is better). **clip_rate**: fraction of ADC outputs saturating at $\pm(2^{b_a-1} - 1)$; high values indicate systematic overflow of the ADC range. **bin_usage**: fraction of the 2^{b_a} output bins actually occupied (higher is better; near-zero means the ADC acts as a 1-bit comparator). **reconstruction_rel**: relative MSE $\|\hat{y} - y^*\|^2 / \|y^*\|^2$ between ADC output and FP reference, tracking layer-level fidelity independently of perplexity.

Optimization. Transforms use AdamW. Static activation scales for the FlatQuant step use 99.9th-percentile clipping. Post-ADC LoRA uses rank 4, LoRA scale 8, KL weight $\lambda = 0.5$, distillation temperature 2.0, and 5 epochs at learning rate 10^{-4} on the same calibration set used for PTQ.

B. Proof of Lemma 1

Proof. Since $P = L \otimes R$ with L, R orthogonal, P is orthogonal, hence $\|xP\|_2 = \|x\|_2$. For any $y \in \mathbb{R}^d$ the standard ℓ^∞ - ℓ^2 inequality gives $\|y\|_\infty \geq \|y\|_2/\sqrt{d}$. Applying this to $y = xP$ yields $\|xP\|_\infty \geq \|x\|_2/\sqrt{d}$, the first inequality. Writing $\|x\|_\infty = \|x\|_2/\sqrt{m_{\text{eff}}(x)}$ and dividing by the lower bound gives

$$\frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \leq \frac{\|x\|_2/\sqrt{m_{\text{eff}}(x)}}{\|x\|_2/\sqrt{d}} = \sqrt{\frac{d}{m_{\text{eff}}(x)}}. \quad \square$$

Corollary 1 (Kronecker capacity). *A full orthogonal matrix on $\mathbb{R}^d$ has $d(d-1)/2$ free parameters, whereas $L \otimes R$ has only $a(a-1)/2 + b(b-1)/2$ for $d = ab$. For $d = 2048 = 32 \times 64$ this is 2512 vs. $\approx 2.1 \times 10^6$. Thus the Kronecker family is a strict low-dimensional subset of all orthogonal transforms and may not attain the bound in Lemma 1; the achievable suppression can be strictly smaller.*

C. Bipolar ADC Ablations

All tables in this section report WikiText-2 perplexity with the signed bipolar ADC ($y_{\text{int}} \in [-M, +M]$) and the constants of Table 3. The FP reference for these calibration runs is ≈ 10.5 .

Table 4 isolates the INT8 calibration objective. Propagation alone reduces ADC perplexity by 47% with no change to dead-rate, confirming that the remaining gap is ADC resolution rather than dead-zone collapse. The bin-center loss is negligible on its own.

Configuration	Bypass	ADC	dead_rate
FP baseline	—	10.5	—
Baseline ($\alpha = 0$)	10.01	28.86	10.3%
Bin-center ($\lambda = 0.1$)	10.01	28.86	10.3%
Propagated ($\alpha = 1$)	11.01	14.55	10.3%
Prop + bin-center	11.00	14.41	10.3%

Table 4: INT8 calibration study with 128 calibration samples and signed ADC step $\Delta \approx 2016$.

Table 5 shows that 128 samples are insufficient for INT4 propagation, while 512 samples with propagation collapse the $\times 153$ gap to $\times 1.2$. Hadamard initialization gives no benefit. Decoupled training (transforms trained for INT8 and evaluated with the INT4 ADC) achieves the best bypass but leaves a poor ADC gap.

Table 6 sweeps α at 1024 samples. $\alpha = 0.5$ gives the best gap ratio; adding the bounded diagonal with $\alpha = 0.5$ gives the best absolute ADC perplexity among pure-PTQ configurations.

Table 7 ablates diagonal placement and decomposes the MLP diagonal: the down projection drives nearly all of the MLP improvement, and staged MLP-then-attention training is the best pure-PTQ checkpoint.

Table 8 compares deterministic $\alpha = 0.5$ with stochastic mixing. Bernoulli switching gives the best bypass but the worst ADC gap; Beta(2,2) matches deterministic mixing without justifying the added complexity.

Config	Smp.	Bypass	ADC	Gap
Baseline	128	15.41	2354.86	$\times 153$
Propagated ($\alpha = 1$)	128	40.28	203.89	$\times 5.1$
Hadamard init + prop	128	34.73	178.86	$\times 5.1$
Decoupled (w8 $\rightarrow$ w4)	128	11.64	276.25	$\times 24$
Propagated ($\alpha = 1$)	512	34.25	40.63	$\times 1.2$
Baseline	1024	19.80	56.83	$\times 2.9$

Table 5: INT4 sample count and propagation sweep with signed ADC step $\Delta \approx 6.12$.

Config	α	Bypass	ADC	Gap
Baseline	0.00	19.80	56.83	$\times 2.87$
Prop	0.25	21.65	32.35	$\times 1.49$
Prop	0.50	24.24	31.22	$\times 1.29$
Prop	0.75	27.46	35.92	$\times 1.31$
Prop (full)	1.00	30.32	40.20	$\times 1.32$
2-stage (A:0, B:0.5)	—	22.82	32.21	$\times 1.41$
Diag + Prop	0.50	20.23	27.56	$\times 1.36$

Table 6: INT4 α -mixing sweep. 1024 calibration samples.

Config	Bypass	ADC	Note
Both diag (control)	19.69	28.46	reproduced
Attn diag only	24.98	29.41	worse bypass
MLP diag only	19.37	31.65	wide gap
MLP diag + LW α	19.00	28.66	best bypass
Both diag + LW α	19.39	28.55	no gain
up_gate diag only	23.22	39.59	weak
down diag only	20.42	31.26	key driver
Staged: MLP $\rightarrow$ attn	18.50	27.60	best PTQ

Table 7: INT4 diagonal placement and staged training. 1024 samples, $\alpha = 0.5$. LW = layer-wise.

Schedule	Mode	Bypass	ADC	Gap
Deterministic 0.5	flat	24.24	31.22	$\times 1.29$
Bernoulli (0.5)	flat	19.25	35.38	$\times 1.84$
Beta(2,2)	flat	22.90	31.40	$\times 1.37$
Deterministic 0.5	staged	18.50	27.60	$\times 1.49$
Bernoulli (0.5)	staged	16.94	32.44	$\times 1.92$
Beta(2,2)	staged	17.16	27.82	$\times 1.62$

Table 8: INT4 stochastic propagation. 1024 samples.

D. Post-ADC LoRA Ablations

Table 9 evaluates LoRA on the best pure-PTQ checkpoint (staged, ADC 27.60). Across all configurations, ADC perplexity falls below bypass (gap < 1), confirming that the adapter specifically corrects ADC-induced error rather than only improving the digital path.

Config	Targets	Bypass	ADC	Gap
No LoRA (base)	—	18.50	27.60	$\times 1.49$
$r = 4$, down	1 proj	19.33	16.32	$\times 0.84$
$r = 8$, down	1 proj	19.17	15.79	$\times 0.82$
$r = 4$, down+o	2 proj	18.68	15.63	$\times 0.84$
$r = 4$, all 7 proj	7 proj	17.73	16.53	$\times 0.93$

Table 9: Initial post-ADC LoRA sweep on the best pure-PTQ checkpoint (bypass 18.50, ADC 27.60).

Table 10 is the full LoRA ablation ($r = 4$, down+o unless stated). Rank saturates at $r = 4$; CE+KL beats CE alone, most strongly with all-7 targets; earlier layers carry more accumulated ADC error; pre-ADC placement diverges. The best single configuration is all-7, CE+KL, $r = 4$ (ADC 14.03), beating INT8 PTQ (14.41).

Axis	Setting	ADC PPL
Seed	seed 1	15.79
	seed 2	15.88
	seed 3	15.93
Rank	$r = 1$	15.90
	$r = 2$	15.60
	$r = 4$	15.57
	$r = 8$	15.24
Loss	CE only	14.60
	CE + KL	13.96
Layer coverage	first 8 layers	16.64
	all 16 layers	15.63
	last 8 layers	20.08
Placement	post-ADC (ours)	15.63
	pre-ADC	105.63 (<i>diverged</i>)
All 7 proj	CE only	16.68
	CE + KL	14.03

Table 10: LoRA ablation study. Llama-3.2-1B, INT4.

Table 11 tests out-of-domain transfer. CE+KL reduces C4 ADC perplexity by 50% relative to PTQ despite WikiText-2-only training, confirming the KL teacher signal is the key ingredient for cross-domain transfer.

E. Unsigned (Unipolar) ADC

Physical crossbars often accept only non-negative currents. Signed INT4 codes $\hat{x}, \hat{w} \in [-8, 7]$ are shifted by the zero-point $z = 8$ to $\hat{x}^u, \hat{w}^u \in [0, 15]$. The crossbar computes one non-negative product $y^u = \langle \hat{x}^u, \hat{w}^u \rangle \in [0, M \cdot 15^2]$,

Config	W-byp	W-ADC	C4-byp	C4-ADC
PTQ (no LoRA)	18.49	27.26	29.41	46.40
$r=8$, d+o, CE	18.51	15.47	30.48	29.39
$r=4$, d+o, CE+KL	19.05	14.27	29.37	23.88
$r=4$, all 7, CE+KL	18.70	13.96	29.10	23.30

Table 11: LoRA generalization. W = WikiText-2, byp = bypass. d+o = down+o.

and the ADC floors it with Δ_u from Eq. (12). The signed inner product is recovered exactly by expanding $\hat{x}^u \hat{w}^u = \hat{x} \hat{w} + 8\hat{w} + 8\hat{x} + 64$:

$$y_{\text{int}} = \hat{y}^u - 8 \sum_j \hat{w}_j^u - 8 \sum_i \hat{x}_i^u + 64M, \quad (13)$$

where $\sum_j \hat{w}_j^u$ is precomputed at weight-load and $\sum_i \hat{x}_i^u$ once per token, adding negligible digital overhead. The per-layer k search selects, for each layer ℓ ,

$$k_\ell^* = \min \left\{ k \in \mathcal{K} : k \geq \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{2R_\ell} \right\}, \quad (14)$$

with $R_\ell = \text{quantile}_{0.999}(|y_\ell^u|)$ and $\mathcal{K} = \{4, 8, 16, 32, 64\}$, run in place on the frozen PTQ checkpoint. FlatQuant is retrained with Δ_u so the transforms are matched to the unipolar step; all other settings carry over from the best bipolar configuration.

The final series was reached through three prior hardware iterations, documented in Table 12. A four-quadrant unipolar decomposition achieves a finer branch step (≈ 3.06) at $4\times$ the ADC reads; a single-read shift-subtract readout is cheaper but harder to calibrate; and an intermediate mismatched setup (transforms trained for the bipolar Δ evaluated on the unsigned model) exploded to $\sim 94k$ perplexity, which motivated retraining with Δ_u . Table 13 reports the final results. Notably, `best_ptq_k_recal` is worse than `best_ptq_k`: retraining FlatQuant on fixed per-layer k over-specializes the transforms, so in-place k search after global- k training is preferable.

Readout setting	Wiki PPL	ADC reads
Signed readout	26.66	1
Four-read unsigned decomp.	18.08	4
Single-read shift-subtract	39.00	1

Table 12: Hardware-setting evolution before per-layer k search and LoRA correction.

F. Full Downstream Results

Table 14 gives the per-task breakdown behind the average downstream column of Table 1.

G. Implementation Sketches and Reproducibility

The listings below are simplified for readability and do not replace the full training code.

Config	Description	Wiki	C4
fp	FP16, no quant.	8.68	13.13
int4_no_adc	INT4 FQ, ADC off	12.56	20.13
best_ptq	+ADC, $k=16$	39.00	67.78
best_ptq_k	+per-layer k	21.64	33.79
best_ptq_k_recal	+FQ retrain, p.l. k	22.19	36.94
best_lora	+LoRA ($k=16$)	17.29	29.26
best_lora_k	+LoRA & p.l. k	14.56	23.78

Table 13: Unipolar (unsigned) ADC results. $\Delta_u = 14.06$ at global $k = 16$.

Listing 1: Tiled ADC forward pass.

```

1 def adc_tile_forward(x, weight, bias,
2   delta, qx, qw, na, pa):
3   sx = x.abs().amax(-1, keepdim=True).
4     clamp(min=1e-6) / qx
5   x_code = round_ste(x / sx).clamp(-qx, qx)
6   sw = weight.abs().amax(1).clamp(min=1e-6) / qw
7   w_code = round_ste(weight / sw[:, None]).clamp(-qw, qw)
8   y_int = linear(x_code, w_code)
9   y_adc = floor_ste(y_int / delta).clamp(na, pa) * delta
10  y = y_adc * sx * sw[None, :]
11  return y if bias is None else y + bias

```

Reproducibility notes. All perplexities use sliding-window evaluation with context length 2048 and stride 1024. The default signed hardware configuration uses tile size 256, ADC bitwidth 8, scaling factor 16, and W4A4 or W8A8 quantization. The main W4A4 PTQ checkpoint uses 1024 WikiText-2 calibration samples with staged diagonal training and $\alpha = 0.5$. Post-ADC LoRA is trained for 5 epochs at learning rate 10^{-4} , rank 4, LoRA scale 8, CE+KL loss, KL weight 0.5, and distillation temperature 2.0.

References

- Anonymous. 2025. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. arXiv preprint.
- Ashkboos, S.; et al. 2024. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. arXiv:2404.00456.

Listing 2: Block-level mixed calibration loss.

```

1 def mixed_block_loss(block_q, x_clean,
2   x_adc, y_teacher, alpha):
3   loss_clean = mse(block_q(x_clean), y_teacher)
4   loss_adc = mse(block_q(x_adc), y_teacher)
5   return (1.0 - alpha) * loss_clean + alpha * loss_adc

```

Method	PPL ($\downarrow$)		Accuracy % ($\uparrow$)								
	Wiki	C4	HSwag	MMLU	WinGr	ARC-E	ARC-C	PIQA	OBQA	BoolQ	Avg
bfloat16	8.68	13.13	64.19	32.15	63.06	61.83	36.77	74.70	37.40	63.91	54.25
INT8 PTQ	8.69	13.15	64.17	31.89	63.38	69.82	39.16	75.79	39.20	64.22	55.95
INT8 + ADC PTQ	16.85	29.56	45.04	26.83	56.59	48.44	28.84	64.36	30.00	59.63	44.97
INT4 PTQ	14.13	22.83	51.98	24.86	55.25	60.44	33.02	69.48	32.00	62.11	48.64
INT4 + ADC PTQ	26.66	45.62	40.03	24.41	53.43	42.63	26.02	61.70	27.00	60.67	41.99
INT4 + ADC + LoRA	14.03	23.20	49.96	25.72	53.67	54.63	29.27	68.06	34.60	60.67	47.07

Table 14: Full downstream results on Llama-3.2-1B. INT8/INT4 PTQ rows have the ADC disabled.

Listing 3: Post-ADC residual LoRA wrapper.

```

1 class PostADCLoRA(Module):
2     def __init__(self, frozen_adc_layer,
3                 in_dim, out_dim, rank=4, alpha
4                 =8):
5         self.base = frozen_adc_layer
6         freeze(self.base)
7         self.A = Linear(in_dim, rank,
8                         bias=False, dtype=float32)
9         self.B = Linear(rank, out_dim,
10                        bias=False, dtype=float32)
11         init_kaiming(self.A.weight);
12         init_zeros(self.B.weight)
13         self.scale = alpha / rank
14
15     def forward(self, x):
16         y_base = self.base(x)
17         y_res = self.B(self.A(x.float()))
18         return y_base + y_res.to(y_base.
19                               dtype)

```

Buechel, J.; et al. 2025. Analog Foundation Models. arXiv:2505.09663.

Chen, M.; Shao, W.; Xu, P.; Wang, J.; Gao, P.; Zhang, K.; Qiao, Y.; and Luo, P. 2024. EfficientQAT: Efficient Quantization-Aware Training for Large Language Models. arXiv:2407.11062.

Ding, X.; Liu, X.; Tu, Z.; Zhang, Y.; Li, W.; Hu, J.; Chen, H.; Tang, Y.; Xiong, Z.; Yin, B.; and Wang, Y. 2023. CBQ: Cross-Block Quantization for Large Language Models. arXiv:2312.07950.

Dubey, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.

Esser, S. K.; McKinstry, J. L.; Bablani, D.; Appuswamy, R.; and Modha, D. S. 2019. Learned Step Size Quantization. arXiv:1902.08153.

Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.

Gao, L.; et al. 2024. A Framework for Few-Shot Language Model Evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>.

Gholami, A.; Kim, S.; Dong, Z.; Yao, Z.; Mahoney, M. W.;

and Keutzer, K. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. arXiv:2103.13630.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Adam, H.; and Kalenichenko, D. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. arXiv:1712.05877.

Le Gallo, M.; et al. 2023. A 64-Core Mixed-Signal In-Memory Compute Chip Based on Phase-Change Memory for Deep Neural Network Inference. *Nature Electronics*, 6: 680–693.

Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, X.; and Han, S. 2023. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978.

Liu, R.; et al. 2024a. FlatQuant: Flatness Matters for LLM Quantization. arXiv:2410.09426.

Liu, Z.; Oguz, B.; Zhao, C.; Chang, E.; Stock, P.; Mehdad, Y.; Shi, Y.; Krishnamoorthi, R.; and Chandra, V. 2023. LLM-QAT: Data-Free Quantization Aware Training for Large Language Models. arXiv:2305.17888.

Liu, Z.; et al. 2024b. SpinQuant: LLM Quantization with Learned Rotations. arXiv:2405.16406.

Merity, S.; Xiong, C.; Bradbury, J.; and Socher, R. 2017. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations*.

Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; and Liu, P. J. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140): 1–67.

Shao, W.; Chen, M.; Zhang, Z.; Xu, P.; Zhao, L.; Li, Z.; Zhang, K.; Gao, P.; Qiao, Y.; and Luo, P. 2023. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. arXiv:2308.13137.

Spall, J.; Guo, X.; and Lvovsky, A. I. 2022. Hybrid Training of Optical Neural Networks. *Optica*, 9(7): 803–811.

Tang, M.; et al. 2024. RAOQ: Reshape and Adapt for Output Quantization. In *International Conference on Machine Learning*.

Wei, X.; Zhang, Y.; Li, Y.; Zhang, X.; Gong, R.; Guo, J.; and Liu, X. 2023. Outlier Suppression+: Accurate Quantization of Large Language Models by Equivalent and Effective Shifting and Scaling. arXiv:2304.09145.

Xiao, G.; Lin, J.; Seznec, M.; Wu, H.; Demouth, J.; and Han, S. 2022. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438.

Xiao, P.; et al. 2021. On the Accuracy of Analog Neural Network Inference Accelerators. arXiv:2109.01262.

Zhang, C.; Cheng, J.; Constantinides, G. A.; and Zhao, Y. 2024. LQER: Low-Rank Quantization Error Reconstruction for LLMs. arXiv:2402.02446.